

PYTHON PROGRAMMING

1. What is Python?

- Python is a high-level, interpreted, and object-oriented programming language.
- It was created by Guido van Rossum and released in 1991.
- Python is easy to learn because of its simple and readable syntax.
- It is widely used in Web Development, AI, Data Science, Machine Learning, Cyber Security, Automation, Cloud Computing and Software Development.

2. Why Python?

- Easy to learn and write.
- Simple and readable syntax.
- Free and Open Source.
- Large community support.
- Huge collection of libraries.
- Platform independent.
- Supports Object-Oriented Programming.
- Used by top companies like Google, Netflix, Microsoft, Instagram, Spotify.

PYTHON PROGRAMMING

3. Features of Python

Feature	Description
Easy Syntax	Python has simple and readable syntax which is easy to learn.
Interpreted Language	Python is an interpreted language, so no compilation is required.
Object-Oriented	It supports OOP concepts like class, objects, inheritance, etc.
High-Level Language	Python is a high-level language, close to human language.
Open Source	Python is free to use and open source.
Portable	Python is platform independent. It runs on Windows, Linux and Mac.
Large Standard Library	Python has a large collection of built-in modules and libraries.
Dynamic Typing	Variables are not required to declare the data type explicitly.
Automatic Memory Management	Python has garbage collection to manage memory automatically.

4. Applications of Python

- Web Development
- Artificial Intelligence (AI)
- Machine Learning
- Data Science
- Cyber Security
- Ethical Hacking
- Automation / Scripting
- Data Analysis
- Game Development
- Desktop Applications
- Cloud Computing

Quick Note :

→ Python = Simple + Powerful + Versatile



PYTHON PROGRAMMING

5. History of Python

- Python was created by Guido van Rossum.
- He started working on Python in December 1989.
- The first version, Python 0.9.0, was released in February 1991.
- Python 2.0 was released in 2000 with many new features.
- Python 3.0 was released in 2008, which is fully new and not backward compatible.
- Python 3.x is the latest and most used version.

Timeline of Python Versions

Version	Release Year
Python 0.9.0	1991
Python 1.0	1994
Python 2.0	2000
Python 2.7	2010
Python 3.0	2008
Python 3.x	Latest (Recommended)

Creator of Python

Guido van Rossum



Born : 31 January 1956
(Netherlands)

6. Advantages of Python

- Easy to learn and use.
- Readable and clean syntax.
- Large community and support.
- Many built-in libraries and modules.
- Platform independent.
- High-level language.
- Supports multiple programming paradigms.
- Increases productivity and reduces development time.

Python is a great choice for beginners as well as professionals.

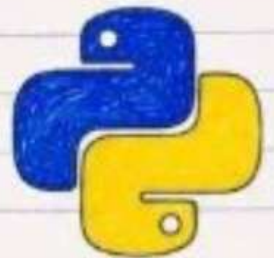
PYTHON PROGRAMMING

7. Installation of Python

1. Go to www.python.org
2. Click on the "Downloads" menu.
3. Download the latest version of Python.
4. Run the installer file.
5. Important: Check the option "Add Python to PATH".
6. Click on "Install Now" and wait for installation.
7. Click on "Close" after successful installation.
8. Verify installation by opening Command Prompt and typing:

```
python --version
```

→ It will show the installed Python version.



8. Python IDEs (Integrated Development Environment)

- IDLE - Default IDE with Python installation. Simple and easy to use.
- PyCharm - Powerful IDE with many advanced features.
- VS Code - Lightweight and supports many extensions.
- Jupyter Notebook - Used for data science and machine learning.
- Spyder - Good IDE for scientific programming.

9. Writing and Running Python Program

Steps:

1. Open any IDE (IDLE / VS Code / PyCharm).
2. Create a new file with .py extension (example: hello.py)
3. Write your Python code.
4. Save the file.
5. Run the program.

Example:

```
# This is a simple Python program
print("Hello, World!")
name = "Monika"
print("Welcome,", name)
```

Output:

```
Hello, World!
Welcome, Monika
```

★ Note: Python is a case-sensitive language.
"Python", "python" and "PYTHON" are different.

10. Variables in Python

- A variable is a container used to store data values.
- In Python, we do not need to declare the type of variable. It is automatically taken care by Python.
- The value of a variable can be changed during program execution.
- Variable name must start with a letter (a-z, A-Z) or underscore (_).
- Variable name cannot start with a number.
- No special characters allowed except underscore (_).
- Python is case-sensitive (age, Age and AGE are different).

Example :

Valid Variable Names ✓	Invalid Variable Names ✗
• name • _name • name123 • Name • age_1 • totalMarks	• 1name (starts with number) • name@123 (special character) • total-marks (hyphen not allowed) • my name (space not allowed) • !age (special character)

Rules for Variable Names

1. Must start with a letter or underscore (_)
2. Can contain letters, digits and underscore only
3. Cannot start with a number
4. No special characters allowed (except _)
5. Case-sensitive

11. Declaring and Assigning Variables

- In Python, we can declare a variable and assign a value in one line.

Example :

```
x = 10           # integer variable
name = "Monika"  # string variable
pi = 3.14        # float variable
is_student = True # boolean variable
```

★ Quick Note

We do not use any keyword like int, float, etc. to declare a variable in Python.

12. Multiple Assignment

- We can assign values to multiple variables in a single line.

Examples :

1. Assigning same value

```
a = b = c = 5
print(a, b, c) # 5 5 5
```

2. Assigning different values

```
x, y, z = 10, 20, 30
print(x, y, z) # 10 20 30
```

3. Using multiple assignment

```
p, q = 100, 200
print(p) # 100
print(q) # 200
```

★ Variables make our programs flexible and reusable. ★

13. Data Types in Python

- Data types specify the type of value a variable can hold.
- Python has the following built-in data types.

Data Type	Description	Example	Type
int	Integer (whole number)	10, -5, 0, 100	int
float	Floating point number	3.14, -0.01, 2.0	float
str	String (sequence of characters)	'Hello', "Python"	str
bool	Boolean value (True / False)	True, False	bool
list	Ordered collection (mutable)	[1, 2, 3, 'A']	list
tuple	Ordered collection (immutable)	(1, 2, 3, 'A')	tuple
set	Unordered collection of unique items	{1, 2, 3}	set
dict	Key-value pair collection	{'name': 'Monika', 'age': 21}	dict

Example :

a = 10	# int	e = [1, 2, 3]	# list
b = 3.14	# float	f = (1, 2, 3)	# tuple
c = "Monika"	# str	g = {1, 2, 3}	# set
d = True	# bool	h = {'name': 'Monika', 'age': 21}	# dict

14. Type Conversion in Python

- We can convert one data type into another data type.
- This process is called type casting.

Common Type Conversions :

Function	Description	Example
int()	Converts to integer	int(3.14) → 3
float()	Converts to float	float(10) → 10.0
str()	Converts to string	str(100) → '100'
bool()	Converts to boolean	bool(0) → False

Example :

a = 5	
b = float(a)	# 5.0
c = str(a)	# '5'
d = bool(a)	# True

★ Understanding data types is very important for writing efficient programs. ★

15. Operators in Python

- Operators are used to perform operations on variables and values.
- Python supports the following types of operators.

Type	Operators	Description	Example
Arithmetic	+, -, *, /, %, //, **	Performs mathematical operations	a + b, a % b
Comparison	==, !=, >, <, >=, <=	Compares two values	a == b, a >= b
Assignment	=, +=, -=, *=, /=, %=	Assigns values to variables	a = 10, a += 5
Logical	and, or, not	Combines conditional statements	a > 5 and b < 10
Bitwise	&, , ^, ~, <<, >>	Performs bit-level operations	a & b, a << 2
Membership	in, not in	Checks membership in a sequence	"x" in str
Identity	is, is not	Checks identity of objects	a is b, a is not b

Example :

```

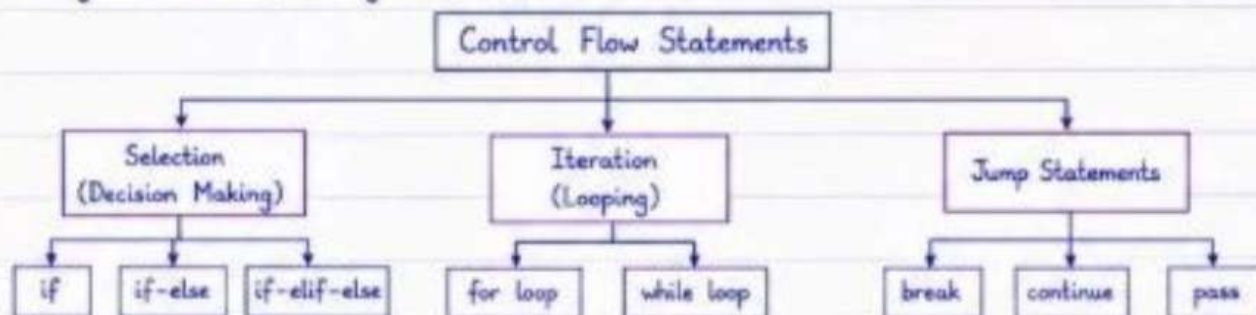
a = 10, b = 3
a + b = 13          # Arithmetic
a > b → True        # Comparison
a += 5 → a = 15     # Assignment
a > 5 and b < 10 → True # Logical
    
```

```

a & b → 2           # Bitwise
"on" in "python" → True # Membership
a is b → False      # Identity
a is not b → True   # Identity
    
```

16. Control Flow Statements

- Control flow statements are used to control the execution of a program.
- Python has the following control flow statements.



Example :

```

1. if statement
x = 10
if x > 5:
    print("x is greater than 5")
    
```

```

2. for loop
for i in range(1, 4):
    print(i)
# Output: 1 2 3
    
```

```

3. while loop
i = 1
while i <= 3:
    print(i)
    i += 1
# Output: 1 2 3
    
```

★ Control flow statements help in making programs more powerful and efficient. ★

17. Functions in Python

- A function is a block of code that performs a specific task.
- It helps to reuse code and make programs more modular.

Syntax :

```
def function_name(parameters):  
    # code to be executed  
    return value
```

Example :

```
def add(a, b):  
    return a + b  
  
result = add(5, 3)  
print(result)      # Output: 8
```

18. Types of Functions

Type	Description	Example
Built-in Function	Predefined functions in Python.	print(), type(), len(), input()
User-defined Function	Functions created by the user.	def my_func(): ...
Anonymous Function	Functions without a name (created using lambda).	lambda x: x * x

19. Modules in Python

- A module is a file containing Python code (functions, variables, classes, etc.).
- It helps to organize code in separate files and reuse it.

Example :

1. Create a file named `mymodule.py`
2. Write some functions in it.
3. Import the module in another program using `import`.

```
# In mymodule.py  
def greet(name):  
    return "Hello, " + name  
  
# In another file  
import mymodule  
print(mymodule.greet("Monika"))  
# Output: Hello, Monika
```

20. Comments in Python

- Comments are used to explain the code.
- Python supports single-line and multi-line comments.

Single-line Comment	Multi-line Comment
<pre># This is a single-line comment print("Hello") # This prints Hello</pre>	<pre>""" This is a multi-line comment """ print("Welcome")</pre>



Functions, modules and comments make the code organized, readable and easy to maintain.



19. Tuples in Python

- A tuple is an ordered, immutable collection of items.
- Tuples are written using parentheses ().

Method	Description	Example	Output
count(x)	Returns number of occurrences of x	t.count(2)	2
index(x)	Returns index of first occurrence of x	t.index(30)	2
len(t)	Returns length of tuple	len(t)	4
max(t)	Returns maximum value	max(t)	30
min(t)	Returns minimum value	min(t)	10

Example:

```
t = (10, 20, 30, 20)
print(t.count(20)) # 2
print(t.index(30)) # 2
print(len(t))      # 4
print(max(t))      # 30
print(min(t))      # 10
```

20. Sets in Python

- A set is an unordered collection of unique items.
- Sets do not allow duplicate values.

Method	Description	Example	Output
add(x)	Adds an element x to the set	s.add(4)	{1, 2, 3, 4}
remove(x)	Removes element x (raises error if not present)	s.remove(2)	{1, 3}
discard(x)	Removes element x (no error if not present)	s.discard(5)	{1, 3}
pop()	Removes and returns a random element	s.pop()	1 (or 3)
clear()	Removes all elements	s.clear()	set()
len(s)	Returns number of elements	len(s)	3
union(s2)	Returns union of two sets	s.union(s2)	{1, 2, 3, 4, 5}
intersection(s2)	Returns common elements	s.intersection(s2)	{3}
difference(s2)	Returns elements in s not in s2	s.difference(s2)	{1, 2}

Example:

```
t = {1, 2, 3}
s.add(4) # {1, 2, 3, 4}
s.remove(2) # {1, 3, 4}
s.discard(5) # {1, 3, 4}
popped = s.pop() # 1 or 3 or 4 (random)
print(s) # remaining set
s.clear() # set()
```

★ Sets are useful when you need only unique items and fast membership testing. ★

21. Dictionaries in Python

- A dictionary is a collection of key-value pairs.
- Dictionaries are mutable and unordered (before Python 3.7).

Method	Description	Example	Output
get(key)	Returns value for key	d.get('name')	'Monika'
keys()	Returns all keys	d.keys()	dict_keys(['name', 'age', 'city'])
values()	Returns all values	d.values()	dict_values(['Monika', 24, 'Indore'])
items()	Returns key-value pairs	d.items()	dict_items([('name', 'Monika'), ('age', 24), ('city', 'Indore')])
update(d2)	Updates dictionary with d2	d.update({'age': 25})	Updated dictionary
pop(key)	Removes key and returns value	d.pop('city')	'Indore'
popitem()	Removes last inserted item	d.popitem()	('age', 24)
clear()	Removes all items	d.clear()	{}
len(d)	Returns number of items	len(d)	3

Example:

```
d = {'name': 'Monika',
     'age': 24, 'city': 'Indore'}
print(d.get('name')) # Monika
print(d.keys())      # dict_keys(['name', 'age', 'city'])
print(d.values())     # dict_values(['Monika', 24, 'Indore'])
print(d.items())      # dict_items([('name', 'Monika'), ('age', 24), ('city', 'Indore')])
d.update({'age': 25})
print(d)              # Updated dict
print(d.pop('city'))  # Indore
print(d.popitem())    # ('age', 25)
print(d)              # {'name': 'Monika'}
d.clear()
print(d)              # {}
```

★ Tuples are immutable, Sets contain unique items and Dictionaries store key-value pairs. All three are important data structures in Python. ★

23. Functions in Python

- A function is a block of code that performs a specific task.
- It improves code reusability, readability and reduces repetition.
- Functions are defined using the `def` keyword.

Syntax :

```
def function_name(parameters):
    statement(s)
    return value # (optional)
```



Quick Note

Functions help in breaking a large program into smaller parts. This makes the program easy to understand and maintain.

Types of Functions

1. Built-in Functions → Provided by Python.
2. User-defined Functions → Created by the programmer.
3. Anonymous (Lambda) Functions → Small, unnamed functions.

Example :

Type	Example Code	Description	Output
1. Built-in Function	<pre>print(len("Python")) print(max(10, 20, 5))</pre>	<code>len()</code> returns length <code>max()</code> returns largest value	6 20
2. User-defined Function	<pre>def add(a, b): sum = a + b return sum print(add(5, 7))</pre>	User-defined function that adds two numbers.	12
3. Lambda Function	<pre>square = lambda x: x * x print(square(6))</pre>	Lambda function to calculate square.	36

Function with Different Types of Arguments

Type of Argument	Example	Explanation
1. Positional Arguments	<pre>def add(a, b): return a + b add(5, 7)</pre>	Values are passed in correct positional order.
2. Keyword Arguments	<pre>add(b = 7, a = 5)</pre>	Values are passed using parameter names.
3. Default Arguments	<pre>def greet(name = "Guest"): print("Hello", name) greet()</pre>	If no value is passed, default value is used.
4. Variable Length Arguments	<pre>def total(*numbers): return sum(numbers) print(total(1, 2, 3, 4))</pre>	Can accept any number of positional arguments.
5. Keyword Variable Arguments	<pre>def info(**data): print(data) info(name = "Monika", age = 21)</pre>	Can accept any number of keyword arguments.

Example Program

```
def multiply(a, b):
    return a * b

x = 6
y = 4
result = multiply(x, y)
print("Result =", result)

Output:
Result = 24
```



Functions make your code modular, reusable and easy to debug.



24. Modules in Python

- A module is a file containing Python code (functions, variables, classes etc.).
- Modules help in organizing code into separate files and promote reusability.
- We can import modules and use their contents in our program.

Quick Note

Python has a large standard library of modules that provide ready-made functionality.

Creating and Using Modules

Steps	Explanation
1. Create a Python file with .py extension.	Example: math_utils.py (This is a module file)
2. Write functions, classes or variables in that file.	Define functions or variables inside the file.
3. Import the module in another program using import.	Syntax: <code>import module_name</code> or <code>from module_name import name</code>
4. Use the functions or variables with module name.	Use with <code>module_name.function_name()</code> or directly if imported.

Example:

File: math_utils.py (Module)

```
def add(a, b):
    return a + b

def square(x):
    return x * x

PI = 3.1416
```

File: main.py (Using Module)

```
import math_utils # importing module
result1 = math_utils.add(5, 7)
result2 = math_utils.square(6)
print("Addition =", result1)
print("Square =", result2)
print("Value of PI =", math_utils.PI)
```

Output:

```
Addition = 12
Square = 36
Value of PI = 3.1416
```

Built-in Modules (Some Common Examples)

Module	Description
math	Provides mathematical functions and constants.
random	Used for generating random numbers.
datetime	Provides classes for manipulating date and time.
os	Provides functions to interact with the operating system.
sys	Provides access to some variables used or maintained by Python.

Importing Modules - Different Ways

Syntax	Example	Explanation
<code>import module_name</code>	<code>import math</code>	Imports the entire module. Use with <code>module_name.function()</code> .
<code>from module_name import name</code>	<code>from math import sqrt</code>	Imports a specific function or variable from module. Use directly by its name.
<code>from module_name import *</code>	<code>from math import *</code>	Imports all names from module. (Not recommended)
<code>import module_name as alias</code>	<code>import numpy as np</code>	Imports module with a shorter alias. Use <code>alias.function()</code> .

Example Programs

1. Using math module

```
import math
r = 5
area = math.pi * r * r
print("Area of circle =", area)
```

Output:

Area of circle = 78.53981633974483

2. Using random module

```
import random
num = random.randint(1, 10)
print("Random number:", num)
```

Output:

Random number: 7 (output may vary)

3. Using datetime module

```
import datetime
now = datetime.datetime.now()
print("Current date and time:")
print(now)
```

Output:

Current date and time:
2024-05-19 14:35:22.123456



Modules make your programs well-structured, organized and easy to maintain.
Always use modules to write clean and reusable code.



25. Packages in Python

- A package is a collection of modules.
- Packages help in organizing related modules in a directory hierarchy.
- Packages are imported just like modules.

Quick Note

Packages help in managing large projects by grouping related modules together.

Creating and Using Packages

Step	Explanation
1. Create a directory (package name).	Example: mypackage/ (This folder is a package)
2. Add <code>__init__.py</code> file in the package directory.	It tells Python that this directory is a package.
3. Add modules (.py files) inside the package.	Example: mypackage/mymodule.py
4. Import and use the modules.	Syntax: <code>from package_name import module_name</code> or <code>from package_name module_name import name</code>

Example

Package Structure:

```
mypackage/
|__ __init__.py
|__ mymodule.py
main.py
```

File: mypackage/mymodule.py

```
def greet(name):
    return f"Hello, {name}!"

def add(a, b):
    return a + b
```

File: main.py

```
from mypackage import mymodule
print(mymodule.greet("Monika")) # Hello, Monika!
print(mymodule.add(10, 20))    # 30
```

Output:

```
Hello, Monika!
30
```

26. File Handling in Python

- File handling allows us to create, read, write, update and delete files.
- Python provides built-in functions to work with files.

Opening a File

Mode	Description	Operation
'r'	Read mode	Opens file for reading. (Default mode)
'w'	Write mode	Opens file for writing. Creates new file or truncates existing file.
'a'	Append mode	Opens file for appending. Creates new file if it does not exist.
'x'	Exclusive creation mode	Creates a new file, returns error if file exists.
'b'	Binary mode	Opens file in binary mode.
't'	Text mode	Opens file in text mode. (Default)
'+'	Update mode	Opens file for both reading and writing.

Common File Methods

<code>read()</code>	Read entire file
<code>read(n)</code>	Read first n characters
<code>readline()</code>	Read one line
<code>readlines()</code>	Read all lines as a list
<code>write(str)</code>	Write string to file
<code>writelines(list)</code>	Write list of lines
<code>close()</code>	Close the file
<code>seek(pos)</code>	Move file pointer to position
<code>tell()</code>	Return current file pointer
<code>truncate()</code>	Truncate the file

Examples

1. Writing to a File

```
f = open("data.txt", "w")
f.write("Hello Python!\n")
f.write("File handling is easy.")
f.close()
```

Output in data.txt:

```
Hello Python!
File handling is easy.
```

2. Reading a File

```
f = open("data.txt", "r")
content = f.read()
print(content)
f.close()
```

Output:

```
Hello Python!
File handling is easy.
```

3. Appending to a File

```
f = open("data.txt", "a")
f.write("\nHave a nice day!")
f.close()
```

Output in data.txt:

```
Hello Python!
File handling is easy.
Have a nice day!
```

4. Reading Line by Line

```
f = open("data.txt", "r")
for line in f:
    print(line.strip())
f.close()
```

Output:

```
Hello Python!
File handling is easy.
Have a nice day!
```



File handling is an important skill in programming.
Always close the file after use using `close()` to free resources.





Quick Note

Good exception handling makes our program robust and user-friendly.

27. Exception Handling in Python

- Exceptions are errors that occur during program execution.
- Exception handling prevents program crash and allows normal flow.
- We use try, except, else, finally blocks.

Syntax

```
try:
    # code that may raise exception
except ExceptionType:
    # code to handle exception
else:
    # code to run if no exception
finally:
    # code that always runs
```

Example

```
a = 10
b = 0
try:
    c = a / b
    print("Result =", c)
except ZeroDivisionError:
    print("Cannot divide by zero!")
else:
    print("Division successful.")
finally:
    print("This block always executes.")
```

Output:

Cannot divide by zero!
This block always executes.

Common Exceptions in Python

Exception	Description	Example
ZeroDivisionError	Raised when division or modulo by zero occurs.	10 / 0
ValueError	Raised when a function gets argument of correct type but inappropriate value.	int("abc")
TypeError	Raised when operation or function is applied to an object of inappropriate type.	"5" * 10
NameError	Raised when a variable or name is not found.	print(x) # x is not defined
IndexError	Raised when index is not found in sequence.	list1 = [1, 2, 3] print(list1[5])
FileNotFoundError	Raised when file or directory is not found.	open("abc.txt")
ImportError	Raised when module cannot be imported.	import unknownmodule

Tips

- Use specific exceptions instead of generic Exception.
- Always use finally block to release resources.
- Keep try block small and focused.
- Provide meaningful error messages.

28. List Comprehension in Python

- List Comprehension provides a concise way to create lists. Examples
- It is faster and more readable than traditional loops.

Basic Syntax

```
new_list = [expression for item in iterable
            if condition]
```

Nested List Comprehension

```
matrix = [[i*j for j in range(1,4)] for i in range(1,4)]
# 3x3 Multiplication Table
Output: [[1, 2, 3], [2, 4, 6], [3, 6, 9]]
```

Example	Output
1. squares = [x*x for x in range(1,6)]	[1, 4, 9, 16, 25]
2. evens = [x for x in range(1,11) if x%2==0]	[2, 4, 6, 8, 10]
3. cubes = [x**3 for x in range(1,4)]	[1, 8, 27]
4. names = ["monika", "patel", "bca"] upper = [name.upper() for name in names]	['MONIKA', 'PATEL', 'BCA']
5. nums = [1, 2, 3, 4, 5, 6] odd_squares = [x*x for x in nums if x%2!=0]	[1, 9, 25]

29. Lambda Function (Anonymous Function)

- Lambda functions are small anonymous functions.
- Defined using lambda keyword. Useful for short operations.

Syntax

```
lambda arguments: expression
```

Examples

Example	Output
1. add = lambda a, b: a + b add(5, 3)	8
2. square = lambda x: x*x square(4)	16
3. even = lambda x: x % 2 == 0 even(10)	True
4. max_num = lambda a, b: a if a > b else b max_num(20, 20)	20

Use in Built-in Functions

```
nums = [1, 5, 2, 9, 3]
# Sort using lambda
nums.sort(key=lambda x: x)
print(nums)
# [1, 2, 3, 5, 9]

# Find max using lambda
maximum = max(nums, key=lambda x: x)
print(maximum)
# 9
```



Exception handling handles errors gracefully.

List comprehension and Lambda functions make code shorter, cleaner and efficient.



30. Map(), Filter() and Reduce()

- These are built-in higher order functions.
- They are used with iterable (like list, tuple, etc.) and a function.
- They help to write clean, short and efficient code.

Quick Note

Map transforms each item, Filter selects items, Reduce combines all items into a single result.

Function	Description	Syntax	Returns
<code>map()</code>	Applies a function to every item of an iterable. (Transformation)	<code>map(function, iterable)</code>	An iterator (map object) with results after applying function.
<code>filter()</code>	Filters items of an iterable that satisfy a given condition. (Selection)	<code>filter(function, iterable)</code>	An iterator (filter object) with items for which function returns True.
<code>reduce()</code>	Applies a function cumulatively to the items of an iterable and returns a single final result. (Aggregation)	<code>reduce(function, iterable[, initializer])</code> (From <code>functools</code> module)	A single value - the final accumulated result.

Examples

1. Using `map()`

```
nums = [1, 2, 3, 4, 5]
def square(x):
    return x * x
squares = list(map(square, nums))
print(squares)
```

Output:

```
[1, 4, 9, 16, 25]
```

2. Using `filter()`

```
nums = [1, 2, 3, 4, 5, 6]
def is_even(x):
    return x % 2 == 0
evens = list(filter(is_even, nums))
print(evens)
```

Output:

```
[2, 4, 6]
```

3. Using `reduce()`

```
from functools import reduce
nums = [1, 2, 3, 4]
def add(x, y):
    return x + y
sum_all = reduce(add, nums)
print(sum_all)
```

Output:

```
10
```

Note

- `map()` and `filter()` return iterators. Convert to list to see results.
- `reduce()` requires at least two elements if no initializer is provided.

31. Classes and Objects in Python

- Class is a blueprint (template) for creating objects.
- Object is an instance of a class.
- Classes help in organizing code and using OOP concepts.

Syntax

```
class ClassName:
    def __init__(self, parameters):
        # constructor
        self.parameter = parameter

    def method_name(self, arguments):
        # method body
        pass

# Creating object
obj = ClassName(arguments)

# Accessing method
obj.method_name(arguments)
```

Example

```
class Student:
    def __init__(self, name, age, roll):
        self.name = name
        self.age = age
        self.roll = roll

    def display(self):
        print(f"Name : {self.name}")
        print(f"Age : {self.age}")
        print(f"Roll No : {self.roll}")

# Creating object
s1 = Student("Monika", 20, 101)

# Calling method
s1.display()
```

Output

```
Name : Monika
Age : 20
Roll No : 101
```

Key Points

- `self` represents the instance.
- `__init__()` is the constructor.
- Attributes store data.
- Methods define behavior.
- Objects are created using class like a real-world entity.

32. Inheritance in Python

- Inheritance allows a class (child class) to inherit attributes and methods from another class (parent class).
- It promotes code reusability and supports hierarchical relationships.

Syntax

```
class ParentClass:
    pass

class ChildClass(ParentClass):
    pass
```

Example

```
class Animal:
    def speak(self):
        print("Animals can speak.")

class Dog(Animal):
    def bark(self):
        print("Dog barks.")

d = Dog()
d.speak() # Inherited method
d.bark() # Own method
```

Output

```
Animals can speak.
Dog barks.
```

Types of Inheritance

- Single Inheritance
- Multiple Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance

33. File and Directory Management

- Python's `os` module provides functions to interact with the operating system.
- We can create, delete, rename files and directories.
- It helps in automating file system tasks.

Common `os` Module Functions

Function	Description	Example
<code>os.listdir(path)</code>	Returns list of files and folders in the given path.	<code>os.listdir(".")</code>
<code>os.getcwd()</code>	Returns current working directory.	<code>os.getcwd()</code>
<code>os.chdir(path)</code>	Changes the current working directory.	<code>os.chdir("C:/Data")</code>
<code>os.mkdir(path)</code>	Creates a new directory.	<code>os.mkdir("NewFolder")</code>
<code>os.rmdir(path)</code>	Removes an empty directory.	<code>os.rmdir("OldFolder")</code>
<code>os.remove(path)</code>	Deletes a file.	<code>os.remove("file.txt")</code>
<code>os.rename(src, dst)</code>	Renames a file or directory.	<code>os.rename("a.txt", "b.txt")</code>
<code>os.path.exists(path)</code>	Checks if path exists.	<code>os.path.exists("test.txt")</code>

Example Program

```
import os

# Current working directory
print("Current Directory:", os.getcwd())

# List files and folders
print("Contents:", os.listdir())

# Create a new directory
os.mkdir("Demo")
print("Created 'Demo' directory")

# Rename a file (assume file 'data.txt' exists)
os.rename("data.txt", "new_data.txt")

# Remove a file (assume file 'temp.txt' exists)
os.remove("temp.txt")

# Remove empty directory
os.rmdir("Demo")

print("Operations completed successfully.")
```

Output (sample)

```
Current Directory: C:\Python\Programs
Contents: ['data.txt', 'notes.txt', 'Demo']
Created 'Demo' directory
Operations completed successfully.
```

34. Working with JSON in Python

- JSON (JavaScript Object Notation) is a lightweight data format.
- It is easy to read and write for humans and machines.
- Python has a built-in `json` module to work with JSON data.

JSON Methods

Method	Description
<code>json.dumps(obj)</code>	Converts Python object to JSON string.
<code>json.loads(str)</code>	Converts JSON string to Python object.
<code>json.dump(obj, file)</code>	Writes Python object as JSON to a file.
<code>json.load(file)</code>	Reads JSON data from a file.

Example

```
import json

# Python object
data = {"name": "Monika", "age": 21, "course": "BCA",
        "skills": ["Python", "DBMS", "Web"]}

# Convert to JSON string
json_str = json.dumps(data, indent=4)
print("JSON String:\n", json_str)

# Write to file
with open("data.json", "w") as f:
    json.dump(data, f, indent=4)

# Read from file
with open("data.json", "r") as f:
    data2 = json.load(f)
    print("Data from file:", data2)
```

Output (sample)

```
JSON String:
{
    "name": "Monika",
    "age": 21,
    "course": "BCA",
    "skills": [
        "Python",
        "DBMS",
        "Web"
    ]
}

Data from file:
{'name': 'Monika', 'age': 21,
 'course': 'BCA', 'skills':
 ['Python', 'DBMS', 'Web']}
```

35. Regular Expressions in Python

- Regular expressions (regex) are patterns used to match character combinations in strings.
- Python has a `re` module to work with regular expressions.

Common Regex Functions

Function	Description
<code>re.match(pattern, string)</code>	Checks for match only at the beginning.
<code>re.search(pattern, string)</code>	Searches for a match anywhere in the string.
<code>re.findall(pattern, string)</code>	Returns all non-overlapping matches.
<code>re.sub(pattern, repl, string)</code>	Replaces matches with repl.
<code>re.split(pattern, string)</code>	Splits string by the pattern.

Example

```
import re

text = "My email is monika123@gmail.com and phone is 9876543210"

# Find email
email = re.search(r'[\w@#\s]+\s', text)
print("Email:", email.group())

# Find all digits (phone number)
phone = re.findall(r'\d+', text)
print("Phone Number:", ''.join(phone))

# Replace digits with *
new_text = re.sub(r'\d+', '*', text)
print("After Replace:", new_text)
```

Output (sample)

```
Email:
monika123@gmail.com

Phone Number:
9876543210

After Replace:
My email is
monika***@gmail.com
and phone is *****
```



Mastering file handling, JSON, and Regular Expressions makes your Python skills powerful for real-world applications like automation, data processing and web development.



Key Takeaway



- Use `os` module for file and directory management.
- Use `json` module to handle structured data efficiently.

- Use `re` to search, validate and manipulate text.
- Practice regularly to build strong programming skills.

36. Working with Date and Time

- Python has a built-in module called `datetime` to work with dates and times.
- It helps in formatting, calculating and manipulating date and time.
- Useful in all types of applications.

Common `datetime` Functions

Function / Class	Description	Example
<code>datetime.now()</code>	Returns current date and time.	<code>datetime.now()</code>
<code>date.today()</code>	Returns current date.	<code>date.today()</code>
<code>strftime(format)</code>	Formats date and time as per given format.	<code>now.strftime("%d-%m-%Y")</code>
<code>strptime(date_str, format)</code>	Converts string to datetime object.	<code>datetime.strptime("25-05-2024", "%d-%m-%Y")</code>
<code>timedelta(days, hours, ...)</code>	Represents difference between two dates/times.	<code>timedelta(days=5)</code>
<code>replace(...)</code>	Returns new datetime with replaced values.	<code>now.replace(year=2025)</code>

Example Program

```
from datetime import datetime, date, timedelta

# Current date and time
now = datetime.now()
print("Current Date & Time :", now)

# Current date
today = date.today()
print("Today's Date :", today)

# Format date
print("Formatted Date :", now.strftime("%d-%m-%Y"))

# Add 10 days to current date
future_date = today + timedelta(days=10)
print("Date after 10 days :", future_date)

# Replace year
new_date = now.replace(year=2025)
print("After replacing year :", new_date)
```

Quick Note

Date and time handling is important in scheduling, logging, reminders, reports and automation systems.

Output (sample)

```
Current Date & Time : 2024-05-25 14:35:22.123456
Today's Date : 2024-05-25
Formatted Date : 25-05-2024
Date after 10 days : 2024-06-04
After replacing year : 2025-05-25 14:35:22.123456
```

37. Virtual Environment in Python

- A virtual environment is an isolated Python environment for a project.
- It helps in managing dependencies separately.
- It avoids version conflicts between different projects.

Common Commands (using `venv`)

Command	Description
<code>python -m venv env</code>	Create a virtual environment named 'env'.
<code>source env/bin/activate</code> (Linux / Mac)	Activate the virtual environment.
<code>env\Scripts\activate</code> (Windows)	Activate the virtual environment in Windows.
<code>deactivate</code>	Deactivate the virtual environment.
<code>pip list</code>	List all installed packages.
<code>pip install package_name</code>	Install a package in the environment.
<code>pip freeze > requirements.txt</code>	Save installed packages to a file.
<code>pip install -r requirements.txt</code>	Install packages from requirements file.

Example: Steps

- Create a virtual environment
`$ python -m venv myenv`
- Activate it
`$ source myenv/bin/activate` (Linux / Mac)
or
`$ myenv\Scripts\activate` (Windows)
- Install packages
`$ pip install numpy pandas`
- Check installed packages
`$ pip list`
- Deactivate environment
`$ deactivate`

Benefits

- Keeps projects isolated.
- Avoids dependency conflicts.
- Easy to share with requirements.txt.
- Professional way of managing projects.

Tip

- ★ Always create and use virtual environment for every new Python project.

38. Debugging in Python

- Debugging is the process of finding and fixing errors (bugs) in code.
- Python provides built-in tools and techniques for debugging.
- It helps to improve code quality and correctness.

Common Debugging Techniques

Technique	Description
<code>print()</code>	Print values to understand the flow.
<code>assert</code>	Check a condition. Raises error if false.
logging module	Provides powerful logging features.
Python Debugger (<code>pdb</code>)	Step-by-step code execution.
breakpoints	Pause execution at a specific line.
try-except	Catch and handle errors gracefully.

Example: Using `pdb`

```
import pdb

def divide(x, y):
    pdb.set_trace() # Start debugger
    return x / y

x = 10
y = 0
result = divide(x, y)
```

Output (sample)

```
(Pdb) p x
10
(Pdb) n
(Pdb) p y
0
(Pdb) n
ZeroDivisionError: division by zero
```

How to use `pdb`?

- Import `pdb` and set `pdb.set_trace()` at the line you want to debug.
- Run the program. The debugger will start.
- Common commands:
n → next line
c → continue
p var → print variable
q → quit debugger

Best Practices

- ✓ Understand the error message carefully.
- ✓ Use `print()` or logging to trace values.
- ✓ Use `pdb` for complex logic errors.
- ✓ Debug small parts of code at a time.
- ✓ Test again after fixing the bug.

Key Takeaway



Practice these advanced topics regularly.
They are essential for real-world Python development!



39. Iterators and Generators in Python

- Iterators are objects that allow you to traverse through all the elements.
- Generators are a simple way to create iterators using functions.
- They are memory efficient and useful for large data.

Iterators

Concept	Description	Example
Iterator	An object that implements <code>__iter__()</code> and <code>__next__()</code> methods.	<code>my_list = [1, 2, 3]</code> <code>it = iter(my_list)</code>
<code>iter()</code>	Returns an iterator object.	<code>iter(my_list)</code>
<code>next()</code>	Returns next value from iterator.	<code>next(it)</code>
<code>StopIteration</code>	Raised when there are no more items to return.	Handled automatically in loops.

Example 1: Using Iterator

```
my_list = ['A', 'B', 'C']
it = iter(my_list)
print(next(it)) # A
print(next(it)) # B
print(next(it)) # C
try:
    print(next(it))
except StopIteration:
    print("No more items")
```

Example 2: Using Generator

```
def square_numbers(n):
    for i in range(n):
        yield i * i

gen = square_numbers(5)
for val in gen:
    print(val)

Output:
0
1
4
9
16
```

Quick Note

- Iterators allow you to iterate over collections like list, tuple, dict, set, etc.
- Generators are created using `yield`.
- Generators are ideal for large datasets or infinite streams.

Generators

Concept	Description	Example
Generator Function	A function that uses <code>'yield'</code> keyword.	<code>def count():</code> <code>yield 1</code>
<code>yield</code>	Pauses the function and returns a value.	<code>yield i</code>
Generator Object	Returned when generator function is called.	<code>g = count()</code>
Memory Efficient	Generates values one at a time, does not store all in memory.	Useful for large data.

Key Difference

Feature	Iterator	Generator
Creation	Built-in / <code>iter()</code>	Function with <code>yield</code>
Memory	Stores all values	Generates one by one
Speed	Slower for large data	Faster and efficient
Use	Simple collections	Large data, pipelines

40. Modules and Packages in Python

- Modules are Python files containing functions, classes, and variables.
- Packages are collections of related modules in a directory.
- They help in organizing large projects.

Creating a Module

```
# mymodule.py
def greet(name):
    return f'Hello, {name}!'

PI = 3.14159
```

Using a Module

```
# main.py
import mymodule
print(mymodule.greet("Nimika"))
print(mymodule.PI)
```

Output:

```
Hello, Nimika!
3.14159
```

Creating a Package

```
mypackage/
├── __init__.py
├── mathops.py
├── stringops.py
└── ...

# mathops.py
def add(a, b):
    return a + b

# stringops.py
def shout(msg):
    return msg.upper()

# Using package
from mypackage import mathops, stringops
print(mathops.add(5, 7)) # 12
print(stringops.shout("hello")) # HELLO
```

Important Points

- Every package must have `__init__.py` file.
- Use dot (`.`) to access modules in a package.
- Packages avoid naming conflicts.
- They improve code reusability and maintainability.

Best Practice

- ✓ Keep modules small and focused.
- ✓ Use meaningful names.
- ✓ Document your modules and packages.

41. Unit Testing in Python

- Unit testing is the process of testing individual components of a program.
- Python has a built-in module called `unittest`.
- It helps in ensuring code correctness and reliability.

Basic Example with `unittest`

```
# test_sample.py
import unittest
import sample # sample.py contains add()

class TestSample(unittest.TestCase):
    def test_add(self):
        self.assertEqual(sample.add(2, 3), 5)
    def test_subtract(self):
        self.assertEqual(sample.subtract(5, 3), 2)

if __name__ == '__main__':
    unittest.main()
```

Sample Module (sample.py)

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b
```

How to Run Tests

- Save both files.
- Open terminal/command prompt.
- Run: `python -m unittest test_sample.py`
- Output will show test results.

Common Assertions in `unittest`

Assertion	Purpose
<code>assertEqual(a, b)</code>	Check if <code>a == b</code>
<code>assertNotEqual(a, b)</code>	Check if <code>a != b</code>
<code>assertTrue(a)</code>	Check if <code>a</code> is True
<code>assertFalse(a)</code>	Check if <code>a</code> is False
<code>assertIs(a, b)</code>	Check if <code>a</code> is <code>b</code>
<code>assertIsNone(a)</code>	Check if <code>a</code> is None
<code>assertIn(a, b)</code>	Check if <code>a</code> is in <code>b</code>
<code>assertRaises(exc, func)</code>	Check if exception is raised



Testing your code regularly leads to fewer bugs and better quality.

Write Tests. Run Tests. Trust Your Code!



Let's Grow Together.
Learn. Build. Stay Consistent.

Yash Dashetwar

Aspiring Data Analyst

SKILLS



SQL



Python



Power BI



Excel



Consistent learning and
practice turns knowledge
into expertise.
Keep Learning, Keep Growing!

LinkedIn
Yash Dashetwar

Follow for daily insights

Keep building. Keep analyzing. Keep growing in your data journey.



More resources coming soon.
Stay Connected!